

บทที่ 2

การเขียนโปรแกรมเน็ตเวิร์กโดยใช้ วินซ็อก(Winsock)

การเขียนโปรแกรมเน็ตเวิร์ก (ทีซีพี/ไอพี) โดยใช้ Winsock

ในบทนี้เราจะนำเสนอการเขียนโปรแกรมผ่านเน็ตเวิร์กที่เชื่อมต่อระหว่างฝั่งไคลเอนต์กับฝั่งเซิร์ฟเวอร์ด้วยการใช้โปรโตคอล ทีซีพี/ไอพี ใน Windows Socket API หรือเรียกสั้นๆว่า Winsock

2.1 ซ็อกเก็ต (socket) คืออะไร

ซ็อกเก็ตถูกนิยามหรือกำหนดไว้ว่าเป็นคู่ของการสื่อสารหรือคู่ของโปรเซส(หรือ เซรด)โดยที่การสื่อสารบนเน็ตเวิร์กใช้คู่ของซ็อกเก็ตสำหรับแต่ละโปรเซสสำหรับซ็อกเก็ตประกอบไปด้วยไอพีแอดเดรสกับหมายเลข Port (Port Number)

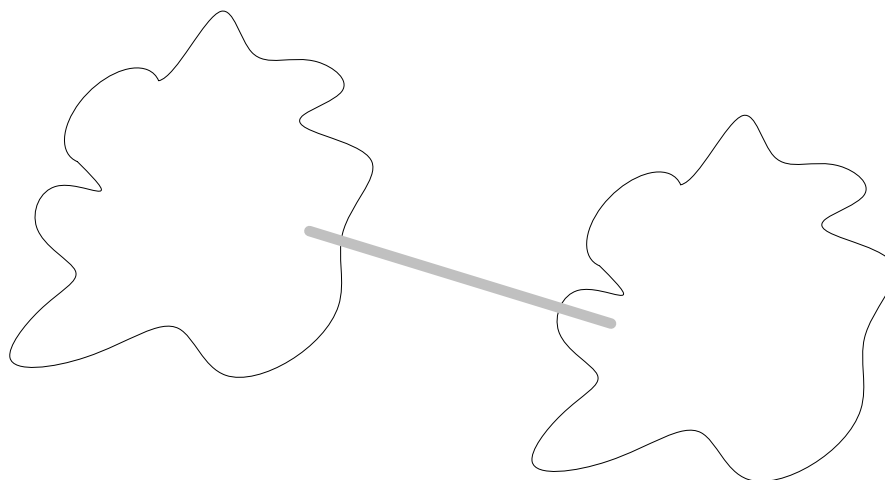
โดยทั่วไป ซ็อกเก็ตใช้สถาปัตยกรรมไคลเอนต์เซิร์ฟเวอร์ เซิร์ฟเวอร์จะรอการเข้ามาตามการร้องขอของไคลเอนต์โดยการฟังที่พอร์ตเฉพาะเมื่อการร้องขอได้รับเซิร์ฟเวอร์ก็จะยอมรับการเชื่อมต่อจากซ็อกเก็ตไคลเอนต์เพื่อให้สมบูรณ์ในการเชื่อมต่อ

เซิร์ฟเวอร์ที่สร้างบริการเฉพาะ เช่น เทลเน็ต, เอฟทีพี, เมล์ และ เอชทีทีพี จะฟัง(Listen)ที่พอร์ตมีชื่อ เช่น เซิร์ฟเวอร์เทลเน็ตจะฟังที่พอร์ต 23, เซิร์ฟเวอร์เอฟทีพีจะฟังที่พอร์ต 21 หรือ เซิร์ฟเวอร์เอชทีทีพีจะฟังที่พอร์ต 80 เป็นต้น

หมายเลขพอร์ตทั้งหมดที่ต่ำกว่า 1024 จะถูกพิจารณาว่าเป็นพอร์ตมาตรฐานที่เราสามารถใช้พอร์ตเหล่านี้เพื่อการสร้างบริการตามมาตรฐานได้

2.2 ตัวอย่างการสื่อสารด้วยซ็อกเก็ต

เมื่อเซรดฝั่งไคลเอนต์เริ่มต้นการขอร้องสำหรับการเชื่อมต่อจะถูกกำหนดพอร์ตโดยโฮสต์คอมพิวเตอร์ (Host computer) พอร์ตนี้เป็นหมายเลขใดๆก็ได้ที่มากกว่า 1024 ตัวอย่าง เช่น ถ้าไคลเอนต์บนโฮสต์ A มีไอพีแอดเดรส 2 ต้องการที่จะสร้างการเชื่อมต่อกับเซิร์ฟเวอร์ เอชทีทีพี (ซึ่งฟังที่พอร์ต 80)ที่มีไอพีแอดเดรส 1 โฮสต์ A จะถูกกำหนดพอร์ต 2544 และที่ฝั่งเซิร์ฟเวอร์จะเป็นพอร์ต 80 สถานการณ์นั้นสามารถแสดงได้ดังรูป แพ็กเก็ตที่ไปมาระหว่างโฮสต์ทั้งสองจะถูกส่งไปยังเซรดที่เหมาะสม ซึ่งขึ้นอยู่กับหมายเลขพอร์ตปลายทางด้วย



IP

รูปที่ 2-1 การสื่อสารโดยใช้ซ็อกเก็ต

การเชื่อมต่อทั้งหมดเป็นคุณสมบัติเฉพาะดังนั้นถ้าโปรเซสอื่น ๆ บนโฮสต์ A ต้องการสร้างการเชื่อมต่ออื่น ๆ กับเซิร์ฟเวอร์เอชทีทีพีเดียวกัน เซิร์ฟเวอร์จะกำหนดหมายเลขพอร์ตที่มากกว่า 1024 และต้องไม่เท่ากับพอร์ต 2544 (เนื่องจากถูกใช้ไปแล้ว) การทำอย่างนี้เพื่อให้แน่ใจว่าการเชื่อมต่อทั้งหมดประกอบด้วยคู่ที่เป็นยูนิก (unique) ของซ็อกเก็ตหรือเป็นสิ่งที่ไม่ซ้ำกับการเชื่อมต่ออื่นๆของซ็อกเก็ต

โดยปกติแล้วเซิร์ฟเวอร์จะมีหลายๆการร้องขอที่พร้อมกันจะต้องใช้ระยะเวลาหนึ่งที่ไคลเอ็นต์ต้องรอคอยเพื่อที่จะถูกบริการโดยเซิร์ฟเวอร์เซรคเดียว ซึ่งจะไม่สามารถรับได้

เพื่อแก้ไขสถานการณ์นี้เซิร์ฟเวอร์ต้องจัดการการร้องขอที่พร้อมๆกันโดยการกำหนดเซรคแยกออกมาเพื่อบริการแต่ละการร้องขอที่เข้ามา ตัวอย่างเช่น เซิร์ฟเวอร์ เอชทีทีพี ที่ไม่ว่าจะกำหนดเซรคแยกออกมาเพื่อบริการแต่ละการร้องขอสำหรับเว็บเพจ

2.3 ชนิดของซ็อกเก็ต

ชนิดของซ็อกเก็ตมีอยู่สามชนิด คือ

- Connection-Oriented Socket
- Connectionless Socket
- Raw Socket

2.3.1 Connection-Oriented Socket

เป็นซ็อกเก็ตสำหรับการเชื่อมต่อแบบต่อเนื่องที่จะยอมอนุญาตให้โปรเซสนั้นๆสามารถเชื่อมต่อกับโปรเซสจากระยะไกล (Remote) ซึ่งใช้โปรโตคอล ทีซีพี (Transmission Control Protocol) ดังนั้นด้วยวิธีการนี้ทำให้ข้อมูลมีความเชื่อถือได้เมื่อการเชื่อมต่อได้เกิดขึ้นโปรเซสก็จะมีการส่งข้อมูลกลับไป

จนกระทั่งฝั่งใดฝั่งหนึ่งหรือฝั่งอื่นๆปิดการเชื่อมต่อชนิดของซ็อกเก็ตแบบนี้บางครั้งเรียกว่า สตรีมซ็อกเก็ต (Stream socket) ทั้งเอพีทีพีและ เอชทีทีพีใช้ซ็อกเก็ตแบบนี้ในการเชื่อมต่อช่องทางการสื่อสาร

2.3.2 Connectionless Socket

ซ็อกเก็ตแบบนี้บางที่จะเรียกว่า เดทาแกรมซ็อกเก็ต ซึ่งซ็อกเก็ตแบบนี้เป็นการเชื่อมต่อแบบไม่ต่อเนื่อง และนำมาใช้ประโยชน์ในการส่งเมสเสจสั้นๆซึ่งไม่สนับสนุนส่วนหัว ดังนั้นจึงพิจารณาการเชื่อมต่อประเภทนี้เป็นแบบเชื่อถือไม่ได้ ซึ่งก็คือ การไม่รับประกันข้อมูลที่ส่งออกไป ไม่เหมือนกับซ็อกเก็ตการเชื่อมต่อแบบต่อเนื่องที่ซ็อกเก็ตปลายทางถูกตรวจสอบเมื่อแพ็กเก็ตถูกส่งออกไป

ซ็อกเก็ตแบบไม่ต่อเนื่อง เปรียบเสมือนกับการบริการของไปรษณีย์ที่ผู้ส่งจดหมายไปตามที่อยู่แล้วใส่ในกล่องรับจดหมาย ผู้ส่งจะไม่ทราบที่ผู้รับได้รับจดหมายหรือไม่ ซ็อกเก็ตแบบนี้นิยมใช้กันในเซิร์ฟเวอร์ DNS (Domain Name System) ที่ใช้ซ็อกเก็ตเดทาแกรมในการตอบสนองต่อการร้องขอที่เข้ามาหลายๆ

นอกจากนี้จะใช้เดทาแกรมซ็อกเก็ตในการกระจาย (Broadcast) เมสเสจ หรือ Multicast เพื่อไปยังปลายทางหลายๆแห่งพร้อมกัน ซึ่งเหมือนกับการกระจายเสียงวิทยุ หรือโทรทัศน์

2.3.3 Raw Socket

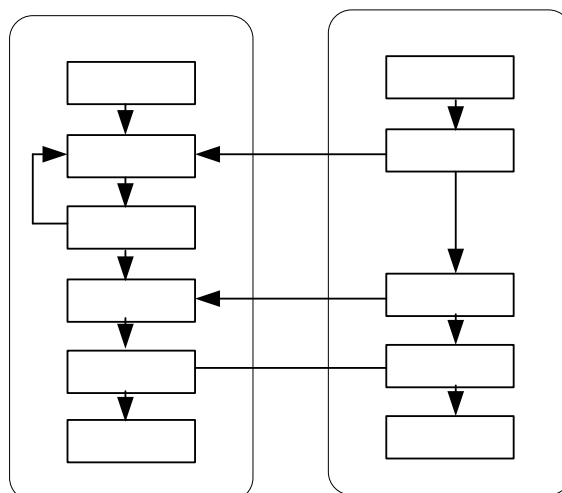
เป็นซ็อกเก็ตที่อนุญาตให้การเข้าถึงโปรโตคอล Transport Raw Socket ยังสามารถนำมาใช้เพื่อจัดการข้อมูลของ “ไอพีเฮดเดอร์” นอกจากนี้แล้วการใช้ซ็อกเก็ตชนิดนี้ยังต้องการความรู้อย่างมากเกี่ยวกับโครงสร้างโปรโตคอลพื้นฐาน

2.4 วินซ็อก (Winsock)

วินซ็อก เป็นมาตรฐานเปิดเน็ตเวิร์ก เอพีไอ โดยที่วินซ็อกถูกออกแบบมาครั้งแรกเพื่อการสร้างโปรแกรมอินเทอร์เน็ตเฟสที่เป็นมาตรฐานสำหรับทีซีพี/ไอพี ในทุกเวอร์ชันของของระบบปฏิบัติการวินโดวส์ รวมทั้ง วินโดวส์ 2000, วินโดวส์ เอ็นที, วินโดวส์ 98 ซึ่งจะเป็นวินซ็อกเวอร์ชัน 2.2 แต่ถ้าเป็นระบบปฏิบัติการวินโดวส์รุ่นดั้งเดิม เช่น วินโดวส์ 95, วินโดวส์ ซีอี นั้นจะใช้วินซ็อกเวอร์ชัน 1.1 มีสองเหตุผลหลักในการใช้วินซ็อก คือ การคอนโทรลและความมีประสิทธิภาพ

วินซ็อก เป็นเน็ตเวิร์กแอปพลิเคชันโปรแกรมมิ่งอินเทอร์เน็ตเฟสไม่ใช่โปรโตคอล ซึ่งวินซ็อกนั้นมีรากฐานเดียวกับซ็อกเก็ตตระกูล บีเอสดี (Berkeley Software Distribution) เวอร์ชัน 4.3 จากมหาวิทยาลัยแคลิฟอร์เนีย วิทยาเขต เบร์เกลีย์ รายละเอียดได้รวมเอาทั้งรูทีนซ็อกเก็ตสไตล์ บีเอสซี (BSC) และการขยายสเปกมาใช้กับวินโดวส์

การใช้วินซ็อกอนุญาตให้แอปพลิเคชันของเราทำการติดต่อสื่อสารข้ามเน็ตเวิร์กใดๆก็ได้ที่กระทำกับ วินซ็อก เอพีไอ แพลตฟอร์ม วิน32 ซึ่งวินซ็อกได้ให้เรดที่ปลอดภัย (thread safty)



รูปที่ 2-2 การเชื่อมต่อการสร้างและการใช้ซ็อกเก็ต

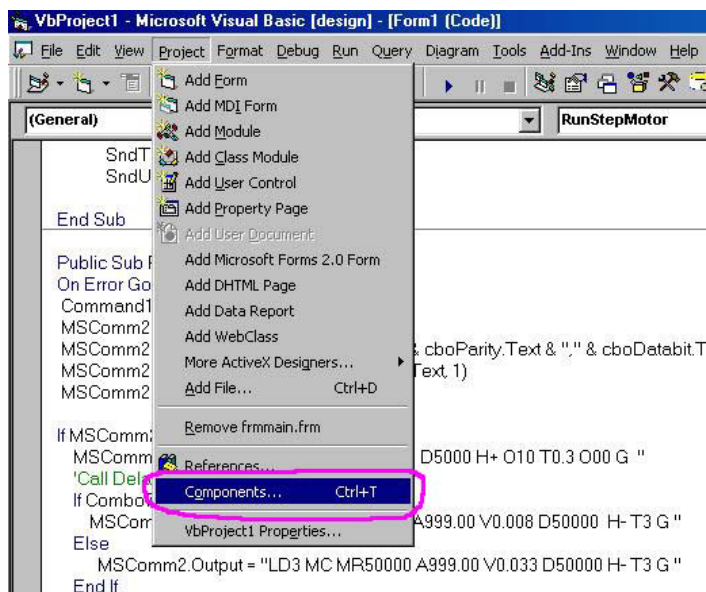
2.4.1 MS Winsock Control 6

เขียนโปรแกรมบนระบบ Windows ให้สามารถติดต่อสื่อสารบนระบบเครือข่ายได้ ถ้าเป็นเมื่อก่อนบนระบบปฏิบัติการ DOS นั้นคงจะยากทีเดียว แต่เดี๋ยวนี้เครื่องคอมพิวเตอร์อะไรก็ Windows กันแล้วเนื่องจากมีเครื่องมือ(Tool)ที่ช่วยให้เราสามารถเขียนโปรแกรมติดต่อผ่านระบบเครือข่าย ที่เราจะกล่าวถึงก็คือ MS Winsock Control 6 เป็นเครื่องมือที่อำนวยความสะดวกสำหรับการเขียนโปรแกรมติดต่อผ่านระบบเครือข่าย โดยโปรโตคอล TCP/IP ซึ่งเราจะทดลองเขียนด้วย Visual Basic หรือถ้าท่านใดถนัด C/C++ ก็ใช้ Visual C++ ท่านใดถนัด ปาสคาล ก็ใช้ Delphi ที่เลือก VB เพราะคิดว่าคงจะอธิบายได้ง่ายแล้ว เข้าใจเร็วซึ่งก็ไม่ซับซ้อนมากนัก ส่วนนอกเหนือจากการที่กล่าวมาแล้วเกี่ยวกับ Winsock Control แล้วยังมี API ที่มีนักพัฒนาเขียนขึ้นมาหลายตัวทีเดียว แต่ดูแล้ว Winsock จะใช้งานง่ายกว่า

จากตอนที่แรกได้กล่าวเนื้อหาเบื้องต้นและโดยรวมเกี่ยวกับการอินเทอร์เน็ตเวิร์กผ่านระบบเครือข่ายโดยมีโปรโตคอล TCP/IP เป็นตัวช่วยในการสื่อสารระหว่างคอมพิวเตอร์ด้วยกันได้ ทั้งนี้ในทางการเขียนโปรแกรมเพื่อติดต่อบนระบบ Windows นั้นก็ได้อำนวยความสะดวกให้เราสามารถที่จะเขียนโปรแกรมติดต่อกับระบบเครือข่ายได้ก็คือ MS Winsock Control ที่มีอยู่ในชุด Visual Studio ในเนื้อหาที่เราจะใช้เขียนโปรแกรมด้วย Visual Basic เพราะจะเข้าใจง่าย, สะดวก, และรวดเร็วต่อการทำความเข้าใจ

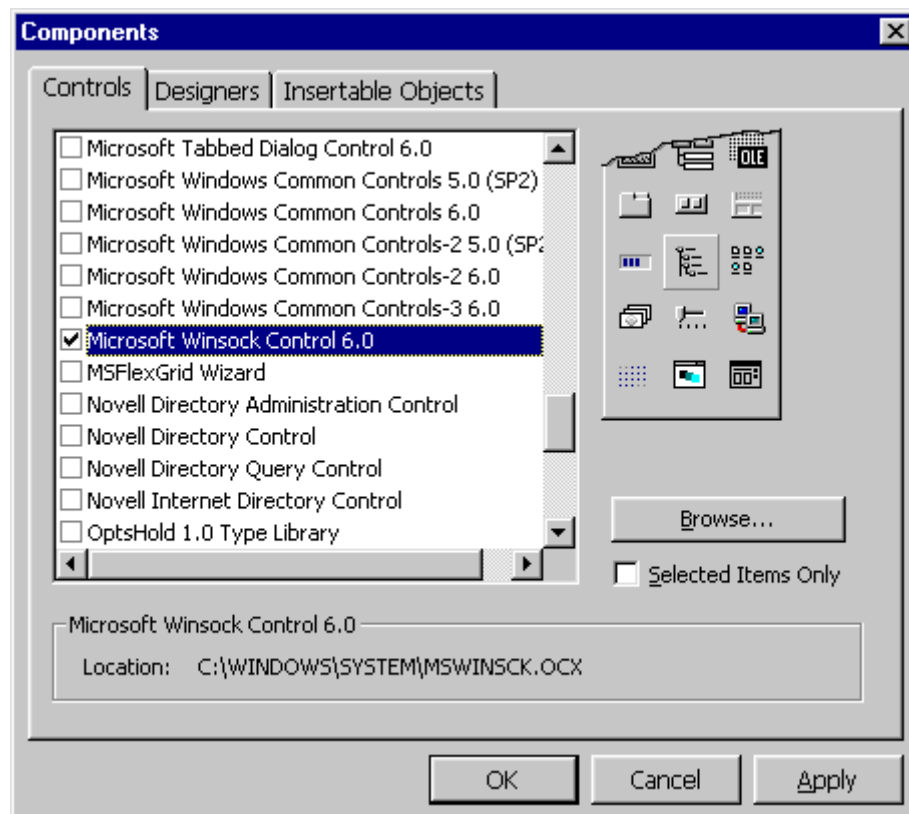
ก่อนอื่นก็เปิดโปรแกรม VB ขึ้นมาแล้วสร้าง Project Standard EXE ขึ้นใหม่ จากนั้นเราจะต้องเพิ่ม Winsock Control ก่อน โดยคลิกที่เมนู

Add Winsock Control

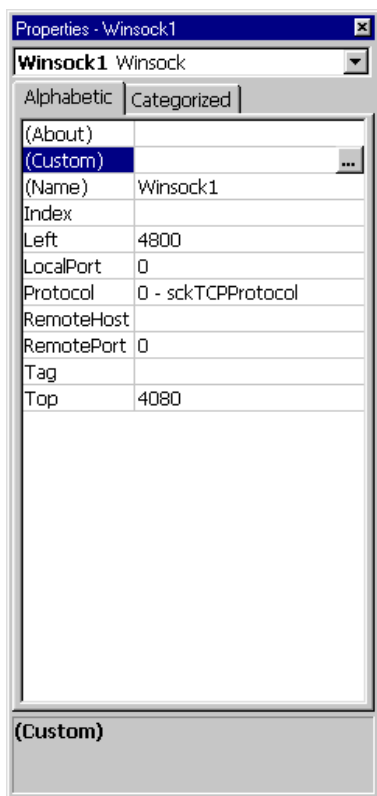


รูปที่ 2-3 แสดงการเพิ่ม components

ให้เลือก Control ชื่อ Microsoft Winsock Control 6.0 (สำหรับ VB6)



รูปที่ 2-4 แสดง Properties Winsock Dialog



Properties	Detail
Name	ชื่อของคอนโทรลที่เราสามารถตั้งได้
LocalPort	Ports ที่เราจะกำหนดในการติดต่อ
Protocol	เลือกชนิดของโปรโตคอลปกติเราใช้ 0-TCPProtocol
RemoteHost	IP Address ของเครื่องที่ใช้ติดต่อ
RemotePort	Port Address ของเครื่องที่ใช้ติดต่อ

Winsock Procedure ในส่วนของ Windows

Code Object --> Winsock มี Procedure สำหรับกระทำติดต่อสื่อสารทั้งทางฝ่าย Server และ Client เราขอกล่าวอย่างนี้เพื่อความเข้าใจที่เหมือนกันดังจะกล่าวในที่นี้คือ

-Close คือ เหตุการณ์เมื่อมีหยุดหรือยกเลิกการติดต่อสื่อสารของฝ่าย Server หรือ Client โดย Function Winsock.Close ซึ่งเราจะสามารถใช้ตรวจสอบฝ่ายตรงข้ามว่ามีการติดต่ออยู่หรือไม่ โดยอาจจะใส่ Message เดือนเป็นต้น

-Connect เป็นเหตุการณ์ที่ฝ่าย Client มีการส่งสัญญาณติดต่อกับมายัง Sever ส่งผลให้ Procedure นี้ของฝ่าย Server ก็เลยทำงานขึ้นมา เหมือนเดิมครับเราสามารถนำ Code Message ไปใส่เพื่อตรวจสอบได้เช่นกัน

-ConnectionRequest เป็นเหตุการณ์เมื่อฝ่าย Client ส่งสัญญาณติดต่อกับมายัง Server Procedure ส่วนนี้จะทำงานพร้อมกับค่า requestID As Long ซึ่งเป็นหมายที่ Gen ขึ้นมาในระบบค่านั้นจะไม่เหมือนเดิม โดยจะให้ฝ่าย Server รับรู้ว่าใช้ ID จากคอนโทรลตัวใดเพื่อจะได้สื่อสารถึงต้อง

-DataArrival เหตุการณ์นี้เกิดขึ้นเมื่อมีการส่งข้อมูลระหว่าง Server และ Client Porcedure นี้ก็จะทำงานขึ้นมา พร้อมกับค่าจำนวน bytesTotal As Long ที่รับเข้ามา

-Error เหตุการณ์ที่เกิดความผิดพลาดระหว่างการติดต่อสื่อสารระหว่าง Server และ Client โดยจะส่งค่า Number As Integer มาให้ว่าเป็นหมายเลขใดพร้อมทั้งรายละเอียดของการผิดพลาดในเหตุการณ์นั้นๆ คือ Description As String

- SendProgress จะเกิดขึ้นในขณะที่มีการส่งข้อมูลอยู่เหตุการณ์นี้จะทำงานเมื่อส่งข้อมูลหมดแล้วก็จะส่งผลทำให้เกิด Event SendComplete

-SendComplete เหตุการณ์เมื่อมีการส่งข้อมูลออกไปยังฝ่ายตรงข้ามเสร็จเรียบร้อยแล้ว

2.4.2 Winsock Properties & Events

Accept (requestID) คือการตกลงกันระหว่าง Server และ Client ในการเลือกหมายเลข ID Control ให้ตรงกันเพื่อสามารถสื่อสารได้ถูกต้อง

Close เป็นการส่งสัญญาณยกเลิกการติดต่อระหว่างกัน จะเป็นฝ่าย Server หรือ Client ก็ได้ ที่จะใช้ Function นี้ จากนั้นจะทำให้ Procedure close ในฝ่ายตรงข้ามทำงาน

Connect เป็นการส่งสัญญาณว่าตอนนี้ทำการติดต่อเรียบร้อยแล้ว ซึ่งจะส่งผลให้ Procedure ฝ่ายตรงข้ามทำงาน

Getdata เป็นการรับข้อมูลเมื่อฝ่ายตรงข้ามส่งมาโดยประโยคคำสั่งนี้จะอยู่ในส่วนของ Procedure DataArrival เนื่องจากเป็นเหตุการณ์ที่การกระทำขณะเมื่อฝ่ายตรงข้ามส่ง ข้อมูลเข้ามา

Listen การกระทำที่จะคอยตรวจสอบสัญญาณที่ส่งไปว่าฝ่ายตรงข้ามตอบรับการร้องขอการติดต่อ

LocalHostName คำสั่งนี้จะส่งชื่อของ Computer name ของเครื่องนั้นๆ

Debug.Print Winsock1.LocalHostName

LocalIP คำสั่งนี้จะทำการส่งหมายเลข IP Address

Debug.Print Winsock1.LocalIP

LocalPort คำสั่งที่จะส่งค่าของหมายเลขในการติดต่อ TCP/IP ของเครื่องนั้นๆ

Debug.Print Winsock1.LocalPort

RemoteHost กำหนดหรือคืนค่าชื่อ Computer name ของเครื่องที่จะทำการติดต่อ

Winsock1.RemoteHost = MyServer

RemoteHostIP กำหนดหมายเลข IP Address ของเครื่องที่จะทำการติดต่อ

Winsock1.RemoteHostIP = 10.10.0.0

RemoteHostPort กำหนดหมายเลข Port ที่จะใช้ในการติดต่อระหว่างกัน

Winsock1.RemoteHostIP = 5000

SocketHandle จะคืนค่าของช่องทางที่ใช้ในการติดต่อระหว่างกันซึ่งสามารถเรียกดูได้ดังนี้

Debug.Print Winsock1.SocketHandle

State จะคืนค่าของสถานะของ Socket ขณะที่ใช้ติดต่อระหว่างอยู่ โดยอาจจะใช้ตรวจสอบสถานะโดยค่าคงที่เหล่านี้เช่น sockClosed (มีค่า=0) Socket ปิดการใช้งาน, sockOpen (มีค่า= 1) Socket เปิดใช้งาน หรือ sockError (มีค่า = 9) Socket มีความผิดพลาดเกิดขึ้น เป็นต้น

2.4.3 Basic Winsock Process Connect

จากที่อธิบายรายละเอียดต่างๆเกี่ยวกับ Winsock control มาบ้างแล้วถึงคราวนี้เราจะมาทำการเริ่มการเขียนโปรแกรมเพื่อที่จะติดต่อกับโพรโทคอล TCP/IP กันละซึ่งการติดต่อกันนั้นจะต้องมีทั้งฝั่ง Server และ Client ซึ่งคล้ายๆจะมี 2 โปรแกรม แต่ความจริงแล้วก็คือโปรแกรมเดียวกันละครับแต่เปิด 2 หน้าต่างโดยกำหนดว่าฝั่งไหนเป็น Server หรือ Client ก็ได้

เหตุการณ์แรกที่ต้องทำในฐานะที่เป็น ฝ่าย Server คือการตรวจสอบสัญญาณจากฝั่ง Client ในที่นี้ใช้ Event Click ของปุ่มชื่อ cmdListen

Private Sub cmdListen_Click()

Winsock1.LocalPort = txtPortSvr.Text

Winsock1.Listen

End Sub

เหตุการณ์ที่สองในฐานะClient ก็จะตอบรับโดยส่งสัญญาณไปให้กับ Server

```
Private Sub cmdConnect_Click()
```

```
Winsock1.RemoteHost ="10.10.0.25"
```

```
Winsock1.RemotePort = "5000" ' กำหนดหมายเลขอื่นก็ได้ แต่ต้องให้ตรงกันทั้ง 2 ฝ่าย
```

```
Winsock1.Connect
```

```
End Sub
```

เมื่อคลิกปุ่ม cmdConnect ตอนนี้ทาง Winsock Control ฝั่ง Sever จะเกิด Event Connect ซึ่งใน Event นี้เราอาจจะใส่ Message ได้อีกได้ดังนี้

```
Private Sub Winsock1_Connect()
```

```
MsgBox "ตอบรับการติดต่อกลับมาแล้ว", vbExclamation, "Chat by MS Winsock Control 6"
```

```
End Sub
```

ขึ้นก็เป็นอันว่าทั้ง Server กับ Client สามารถติดต่อกันได้แล้ว เ้าลองส่ง Data กันหน่อยสิ

```
Private Sub cmdSend_Click()
```

```
Winsock1.SendData "Hi! How are you ? "
```

```
End Sub
```

เมื่อฝ่าย Server ส่งData ออกไปที่ Winsock control ฝั่ง Client จะเกิด Event DataArrival เราก็จะรับข้อมูลนั้นด้วยคำสั่ง Getdata โดยเขียนโปรแกรมไว้ใน Event นี้

```
Sub Winsock1_DataArrival(ByVal bytesTotal As Long)
```

```
Dim StrValue As String
```

```
Winsock1.GetData StrValue
```

```
End Sub
```

ถ้าหากต้องการยกเลิกการติดต่อก็ใช้คำสั่ง Close เมื่อ Server หรือ Client ใช้คำสั่งนี้ จะทำให้เกิด Event Close ขึ้นกับ Winsock control ของฝั่งนั้น ดังเช่นเราสร้างปุ่ม Close ไว้

```
Private Sub cmdClose_Click()
```

```
Winsock1.SendData "ยกเลิกการติดต่อแล้วครับ"
```

```
Winsock1.Close
```

```
End Sub
```

สมมุติว่าเมื่อฝั่ง Server มีการคลิกปุ่มเกิดขึ้นก็จะทำให้ Winsock Control ฝั่ง Client เกิด Event Close โดยอาจจะเขียน code Message ได้อีกได้ดังต่อไปนี้

```
Private Sub Winsock1_Close()
```

```
    MsgBox "ยกเลิกการติดต่อแล้วครับ", vbExclamation, "Chat by MS Winsock Control 6"
```

```
End Sub
```